Chapter 1
Introduction to C

CSE 4107 : Structured Programming I
REYNEP

1.1.1. History of C : Origins

C is a by-product of the UNIX. UNIX was 1 of the first Operating Systems, developed at Bell Laboratories by Ken Thompson, Dennis Ritchie, and others. Thompson single-handedly wrote the original version of UNIX, which ran on the DEC PDP-7 computer, an early minicomputer with only 8K words of main memory (this was 1969, after all!).

Like other operating systems of the time, UNIX was written in assembly language. Programs written in assembly language are usually painful to debug and hard to enhance; UNIX was no exception. Thompson decided that a higher-level language was needed for the further development of UNIX. So, Thompson designed a small language named B based on BCPL, a systems programming language developed in the mid-1960s. BCPL, in turn, traces its ancestry to Algol 60, one of the earliest (and most influential) programming languages.

Ritchie soon joined the UNIX project and began programming in B. In 1970, Bell Labs acquired a PDP-11 for the UNIX project. Once B was up and running on the PDP-11, Thompson rewrote a portion of UNIX in B. By 1971, it became apparent that B was not well-suited to the PDP-11, so Ritchie began to develop an extended version of B. He called his language NB (“New B”) at first, and then, as it began to diverge more from B, he changed the name to C. The language was stable enough by 1973 that UNIX could be rewritten in C. The switch to C provided an important benefit: portability. By writing C compilers for other computers at Bell Labs, the team could get UNIX running on those machines as well.

1.1.2. Chronological evolution of C language



1.1.3. History of C : Standardization (page 1)

C continued to evolve during the 1970s, especially between 1977 and 1979. It was during this period that the first book on C appeared. The C Programming Language, written by Brian Kernighan and Dennis Ritchie and published in 1978, quickly became the bible of C programmers. In the absence of an official standard for C, this book—known as K&R or the “White Book” to aficionados—served as a de facto standard.

During the 1970s, there were relatively few C programmers, and most of them were UNIX users. By the 1980s, however, C had expanded beyond the narrow confines of the UNIX world. C compilers became available on a variety of machines running under different operating systems. In particular, C began to establish itself on the fast-growing IBM PC platform.

With C’s increasing popularity came problems. Programmers who wrote new C compilers relied on K&R as a reference. Unfortunately, K&R was fuzzy about some language features, so compilers often treated these features differently. Also, K&R failed to make a clear distinction between which features belonged to C and which were part of UNIX. To make matters worse, C continued to change after K&R was published, with new features being added and a few older features removed. The need for a thorough, precise, and up-to-date description of the language soon became apparent. Without such a standard, numerous dialects would have arisen, threatening the portability of C programs, one of the language’s major strengths.

1.1.3. History of C : Standardization (page 2)

The development of a U.S. standard for C began in 1983 under the auspices of the American National Standards Institute (ANSI). After many revisions, the standard was completed in 1988 and formally approved in December 1989 as ANSI standard X3.159-1989. In 1990, it was approved by the International Organization for Standardization (ISO) as international standard ISO/IEC 9899:1990. This version of the language is usually referred to as C89 or C90, to distinguish it from the original version of C, often called K&R C. Appendix C summarizes the major differences between C89 and K&R C.

The language underwent a few changes in 1995 (described in a document known as Amendment 1). More significant changes occurred with the publication of a new standard, ISO/IEC 9899:1999, in 1999.
The language described in this standard is commonly known as C99. The terms “ANSI C,” “ANSI/ISO C,” and “ISO C”—once used to describe C89—are now ambiguous, thanks to the existence of two standards.

Because C99 isn’t yet universal, and because of the need to maintain millions (if not billions) of lines of code written in older versions of C, I’ll use a special icon (shown in the left margin) to mark discussions of features that were added in C99. A compiler that doesn’t recognize these features isn’t “C99-compliant.” If history is any guide, it will be some years before all C compilers are C99-compliant, if they ever are. Appendix B lists the major differences between C99 and C89.

CURRENT STANDARD ⇒ C11 ⇒ Yeah, you guessed it! (2011, C-Standard)

1.1.3. History of C : Standardization (page 3) `ivan's lines`

List of Standards

1. *K&R “de facto White Book” —--> 1978*
2. *C89/C90/K&R C —---> ANSI* standard approved in 1989*
3. *ANSI C/C99 —--> New standard in 1999 by ANSI*
4. *Current Standard is : C11*

*ANSI (American National Standards Institute)

1.1.4. History of C : C-Based Languages `ivan's lines`

- `C++` *includes all the features of C, but adds feature support for object oriented programming*
- `Java` *based on* `C++` *and therefore inherits many features*
- `C#` *is more recent language derived from* `Java` *and* `C++`
- `Perl` *another scripting language, adopted many features of* `C`
- `JavaScript` *uses syntax influenced by* `C`

1.2. Strength & Weaknesses of C `ivan's lines`

- Low level language
  - *Provides access to machine level concepts*
- Comparatively small
  - *C keeps itself limited by relying heavily on a library of standard functions*
- Permissive
  - *Provides more freedom compared to other higher level languages (which is also more risky)*